

なぜPythonか

2021-03-04 ITロボット塾/ITゲームクラス担当 辻 直弘

ITロボット塾、ITゲームクラスでは、Scratchで入門、3年目、4年目でPythonへ橋渡しというカリキュラムで授業を進めています。なぜ、Scratchなのか。なぜPythonなのでしょう？特に、C言語、C++/C#、Java、Javascript、Ruby、PHPなど数あるプログラミング言語から、なぜPythonなのか。このことを中心に説明します。

- Pythonとは、どういうプログラミング言語か
- 使い慣れたScratchで試し、同じことをPythonで取り組む
- 手に職、としてのプログラミング言語？
- プログラミング言語は、一生にひとつだけ？
- 構築主義とScratch、Python

Pythonとは、どういうプログラミング言語か

Pythonは、誕生から30年のプログラミング言語。一番古いFORTRANから並べてみるとこんな感じ。FORTRAN（67歳）、LISP（63歳）、COBOL（62歳）、C言語（49歳）、C++（38歳）、Javascript（26歳）、Ruby（26歳）、Java（25歳） 中堅といったところ？

普通は言語仕様として組み込まれてしまいがちな機能（関数／モジュール）を極力外部に押し出し、プログラミング言語として簡素で理解しやすい状態を保っていることが特徴のひとつ。そして、外部モジュール／ライブラリを必要に応じて読み込み、Python製、C言語製を意識せずに使えることから、高速、高機能なモジュール／ライブラリが数多く存在し、応用プログラムの開発例も多い。プログラミングの専門家ではない研究者にも比較的扱いやすいことから、近年では、機械学習、画像処理、統計／ビッグデータ解析などの分野において欠かせない存在となっている。

結果として、最先端の分野での利用者、入門者が多い状況となっており、プラットフォーム開発の競争もさかんに行われるなど、好循環が生まれている。

使い慣れたScratchで試し、同じことをPythonで取り組む

さらに、組み込みなどのハードウェアを扱う分野においても、処理速度を要求される部分にはC言語製のモジュールを使い、マン・マシンインターフェースや処理モデルなどの構築にPythonを使うことで、高速動作を可能にしながら、バグ修正、仕様変更、などにも迅速に柔軟な対応ができるシステムを作ることができる。

また、インタープリタとしての動作環境（iPython、REPL、Jupyter Notebook）が、Scratchプログラミングとの共通性を高めており、初学者にも優しい。コードの断片をチェックしやすく、ブロック単位の組み合わせ、取替え、すなわち、試行錯誤が容易、というイメージ。C言語（C++/C#）やJavaなどとの大きな違いである。今まで知らなかった言語仕様や関数、データ構造、アルゴリズムなどを試す過程で、新しい概念／知識の獲得をすすめることができ、自発的に学ぶ学習方法に適している。

ITゲームクラスでは、ゲーム制作、タートルグラフィックス、マインクラフト3D世界でのプログラミング、ネットワーク通信活用、ハードウェア制御、などの様々な題材において、まず使い慣れたScratchで試し、概念を獲得／構築しつつ、Pythonでの挑戦につなげるカリキュラムが完成してきている。

手に職、としてのプログラミング言語？

では、手に職をつける、対価をもらう仕事／作業としてのプログラミングについては、どうか。Pythonだけやっていたら良いのか？結論から言うと、Pythonは、向いていない。

実際にお金が動くのは、基幹システム、バックエンドのJava。ウェブサイトのフロントエンド（対人インターフェース）としてのJavascript。デザインとミックスの場面でのPHP。組み込み系の現場でのC言語／C++/C#。スマホアプリのSwift（iPhone/iPad）、Java（Android）。Pythonは、ウェブアプリの分野でもサーバーサイド／フロントエンド双方で使われているが、とても主流とは言えない。

ウェブアプリなら、Javascriptの伸びが激しい。元々は、ブラウザと共に動くクライアントサイドの処理系であったが、サーバーサイドでも動くようになり、さらには、ハードウェアにもアクセスできるようにもなったため、全てをJavascriptで、という方向性が強まった。開発ツールの発達により、デザインと深く結びついた形でデザイナー／プログラマーが活躍できる環境が整ってきたと言える。

組み込み系なら、C/C++。部品コストを低減し、コンピューターの処理速度を上げる（この2つは背反条件）必要がある組み込み系においては、全て、C/C++で書く、さらには、アセンブラ（機械語）で書く場合がある。しかし、開発コスト、メンテコストなどを含めると、今後の技術進化に伴う基本的な処理能力向上によって、JavascriptやPythonなどの「高級言語」が使われる可能性も出てくる。組み込み系以外では、実際にそういう経緯で高級言語の採用が進んでいる。

プログラミング言語は、一生にひとつだけ？

プログラミング言語は、宗教のように一度入信すれば、一生、というものではない。ITロボット塾では、プログラミング初学者にとっての最初の一步、二歩を考えている。筋が良いもの。その先で応用が効くもの。

最初の一步は、Scratch。次は、Python。その次に来るものは、C/C++とJavascript。

Scratch / Pythonの特徴、向いていること：

- ・ 試行錯誤。ちょっと試して、修正、の素速い反復をすること。
 - ・ 小さなブロックから組みはじめること。
 - ・ ブロックを組み合わせること、組み替えること。
 - ・ ブロックを組み合わせで大きなシステムを構築すること。
 - ・ 言語仕様自体を理解しやすいこと。
- 結果として、初学者に向いている。新しい学びのツールとして自学自習に向いている。

構築主義とScratch、Python

ピアジェの構成主義、それを引き継いだパパートの構築主義。パパートとミッチェル・レズニックによるマインドストームロボットとScratch。ピアジェとパパートについては、別文書を参照のこと。ここでは、レズニックのMITメディアラボ／ライフロンギンダーガートン研究室に至るまでの流れを見てみよう。

1837年 フレーベル、幼稚園を「発明」
 1907年 モンテッソーリ、教育概念を提唱
 1934年 レゴ、ブロックを発売
 1967年 パパート、LOGO言語（タートルグラフィクス）を作る
 1985年 パパート、レズニック、レゴ社と共同開発開始
 1998年 マインドストームロボット発売
 2006年 レズニック、Scratchをリリース

いずれも、モノと触れ合う（タンジブル）感覚を大切にしている。

Pythonは、この流れとは接点がないが、Python創始者ガイド・ヴァンロッサムは、初学者向けにも良いという趣旨の発言をしたことがある。ちなみに、ヴァンロッサムは、1956年オランダ生まれ、アムステルダム大学卒、オランダ、アメリカの政府系研究機関に勤務の後、2005年Google入社、2012年Dropbox社へ移籍、2018年引退宣言、2020年引退から復帰、マイクロソフトへという経歴の持ち主、現在、65才。

GoogleでPythonが多用されていること、TensorFlowという、機械学習プラットフォームがPythonで書かれていること、Google Colaboratoryの存在、マイクロソフト製の開発環境VS CodeでのPython / Jupyter Notebookの扱いの良さは、全くの偶然ではないだろう。

MITメディアラボでは、プログラマーを量産するためのツールとしてScratchやマインドストームを開発したのではなく、新しい表現のためのツール、新しい学びのためのツールなのだ、と言う。ITゲームクラスでも、「プログラマー養成」とは、全く違うアプローチで進めている。最先端／即戦力は鋭い刃物と同じで、すぐに切れなくなってしまうから。技術的なスキルを付け焼き刃で付けることよりも、むしろ、（全く）新しいことに、どう自発的に粘り強く取り組むのか、といった態度／姿勢を育みたいと考えている。